Aurelia: CXL Fabric with Tentacle

Shu-Ting Wang
shw328@ucsd.edu
UC San Diego
California, USA

Weitao Wang
wtwang@rice.edu
Rice University
Texas, USA

ABSTRACT

Compute Express Link (CXL) has emerged as a frontier for disaggregation by providing a fabric supporting memory accessing, caching, and peer-to-peer memory access between devices. CXL externalizes the internal memory fabric of a server and blurs the notion of server for realistic disaggregation. The key feature of enabling of CXL as a memory fabric is its support of multi-level switching up to 4096 endpoints. However, CXL's current multi-level switching poses challenges on scalability and latency. To cope with the expected scale of CXL fabric and take full advantage of disaggregation, we propose Aurelia. Aurelia architects addressing, routing, and transport as networking layers, which are typical on host networking, for CXL fabric. Aurelia uses existing CXL mechanism to realize the much-needed functionalities for a scalable fabric. With these networking layers, Aurelia will be able to support future disaggregation workloads at scale.

CCS CONCEPTS

• **Networks → Network architectures**; **Network algorithms**; **Network protocols**.

ACM Reference Format:
Shu-Ting Wang and Weitao Wang. 2023. Aurelia: CXL Fabric with Tentacle. In *4th Workshop on Resource Disaggregation and Serverless (WORDS '23), October 23, 2023, Koblenz, Germany*. ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3605181.3626287

WORDS '23, October 23, 2023, Koblenz, Germany

ACM ISBN 979-8-4007-0250-1/23/10…$15.00
https://doi.org/10.1145/3605181.3626287

1 INTRODUCTION

The datacenters move towards disaggregated and composable infrastructure, in which different resources are taken from a logical pool to satisfy the demand from applications. Existing effort retrofits the current Ethernet fabric for disaggregation by running RDMA over Ethernet [17, 32, 36, 47, 48, 55, 57]. However, retrofitting existing hardware prevents applications from reaching their optimal performance under disaggregation and sometimes demonstrates inferior performance than its counterpart with server-based setups.

Recent works look toward co-designing hardware with software to realize disaggregation [20, 33, 37, 50]. These works are focused on externalizing the hardware interconnect to become a fabric connecting many devices in a disaggregated setting. Fabrics directly connect all the devices allowing accessing remote devices to be similar to accessing a local device on a server's PCIe slots. PCIe is an example of this kind of fabric but it only connects local devices within a server right now. An ideal fabric for disaggregation offers direct connections between devices while providing low latency and high bandwidth at scale.

Compute Express Link (CXL) emerges as a viable candidate supported by a converged industry standard after its absorption of OpenCAPI and Gen-Z. CXL is built on top of PCIe with the addition of memory accessing semantics (CXL.mem), caching semantics (CXL.cache), and peer-to-peer memory access between devices (Unordered I/O in CXL.io). More importantly, CXL can be a fabric through its support of multi-level switching.

Experimental CXL fabric demonstrates on-par performance with lower cost in the case of machine learning model training on 10s of GPUs [10]. CXL fabric offers bandwidth as high as 63 GB/s with PCIe 5.0 now and 121 GB/s PCIe 6.0 on the horizon of the next two years [14]. CXL fabric offers low latency because it operates on a device-attached interface using direct load/store instructions. It avoids the network software stack overhead and the PCIe transition between device and NIC on the sending (Device → PCIe → NIC) and receiving (NIC → PCIe → Device) path. Network stack and PCIe transition create latency overhead and throughput bottleneck.

First, the network stack using kernel bypassing still incurs microseconds of latency overhead [26, 27, 35, 42]. Second, the PCIe latency overhead of 1500 B packets reaching the wire can be as high as 77% [40]. Third, the PCIe link to NIC is a potential throughput bottleneck when multiple devices share the NIC. Dedicating a NIC for each device circumvents the throughput bottleneck but with the increased cost on more NICs and switches.

2 MOTIVATION

2.1 What is Different with CXL Fabric?

CXL fabric exposes memory traffic that used to be internal to a server to all endpoints connecting on the fabric. The memory traffic, such as cache coherence, memory access, and I/O style accesses, runs between processor and device endpoints on CXL fabric. This is in contrast to standard data center traffic running with encapsulated packets from per-server NIC outside of the internal memory fabric of a server. Processors and accelerators access remote devices with load/store instructions through CXL fabric. They synchronously request data from remote devices, e.g. memory expansion modules, accelerators, and storage devices. However, these synchronously request data cannot tolerate significant latency, because the latency will stall the execution of the requester hardware while awaiting the requested data. This poses restricted latency requirements for the fabric and needs proper system-level support. Additionally, CXL fabric allows up to 4096 endpoints. Given the scale of 1000s of endpoints and the mixture of memory traffic, this introduces a challenge on the scalability of the underlying protocol design. To understand the practical challenges, use cases of CXL fabric from CXL specification and the literature are discussed next [3, 24, 33].

2.2 Use Cases of CXL fabric

The use cases of CXL fabrics demand high memory capacity, bandwidth, and low latency. Emerging and existing workloads in data centers, such as training machine learning models, high-performance computing (HPC) applications, and large-scale key-value stores, can be benefited from CXL fabrics.

First, training machine learning models requires a large amount of memory on an accelerator, e.g. GPU or TPU, which is beyond the capacity of individual accelerators. To make it even worse, the size of state-of-the-art machine learning models are ranged from 10s of GB to 10s of TB. [39, 45, 46] and keep growing every few months. Model training thus requires multiple accelerators to jointly fit the model and training variables in their memory. CXL fabric expands accessible memory to accelerators by providing fabric-attached memory. Fabric-attached memory increases the memory capacity and bandwidth to all available memory on the fabric [3, 7, 43]. A host is connected with an accelerator with CXL and they share a coherence domain (Shown in Figure 1a). Accelerators access the fabric-attached memory and each other's memory in a producer-consumer fashion of I/O coherency.

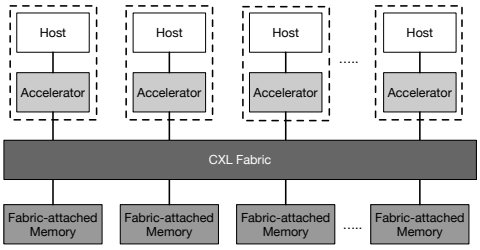
Second, HPC workloads demonstrate high utilization (> 90%) on memory bandwidth as well as capacity for representative applications [1, 2, 8]. Though each application stays with its peak memory usage for a different duration of time. The cluster is required to be provisioned to the peak bandwidth and capacity to avoid significant slowdown [22, 25, 44].

Third, the datacenter runs cloud services with key-value stores. Key-value stores use RDMA inside datacenter to speedup the communication and operations [23, 27, 28, 56] These operations are sensitive to latency and are on the performance critical path of applications. DirectCXL showed CXL to be 8.3x shorter latency than RDMA for 64B read and replacing RDMA with less overhead [24]. DircetCXL provides a latency lower bound of CXL because its fabric has a single switch and is not under stress or congestion. The hosts do not maintain cache coherence between themselves. Fabric-attached memory module maintains coherence between itself and the host address space it has mapped (Shown in Figure. 1b).

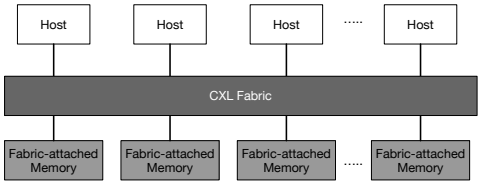
Model training and HPC workload both involve collective communication while key-value stores involve bursty non-structural communication. Collective communication is structural and can be optimized with data prefetch to minimize the stall on pending memory accessing. This relaxes their requirement on latency and reduces burstiness. HPC compared to model training has a more diverse communication pattern, such as sweeping or nearest-neighbors [1, 8, 9, 31]. Key-value store and database, however, serve bursty requests and are sensitive to latency [21, 41]

3 CHALLENGES

The current design of CXL fabric [3] poses challenges on scalability and latency. First, its addressing and routing design limits the possibility of flexible and dynamic routing. Second, the lack of an end-to-end transport layer of the fabric makes the fabric prone to congestion and latency spikes. More importantly, with the usage of load/store instructions, the processor and accelerator that synchronously request data are very sensitive to



(a) Model training and HPC. Hosts and their accelerators share a coherent domain.



(b) Key-value store for cloud services.

Figure 1: CXL fabric abstractive topology.

latency because the latency determines how long they need to stall their execution. The addressing, routing, and transport-level challenges are discussed in the following subsections.

3.1 Addressing and Routing Challenges

Routing. CXL routes packets with a per-device ID called Port ID on the fabric. The Port ID-based routing (PBR) addresses each device with a 12-bit ID. A packet with PBR contains a specific source port ID and destination port ID before it leaves an edge CXL switch that connects directly with devices. Each CXL fabric has a single fabric manager to initialize, bind/unbind devices to ports, and handle event notifications, such as the removal or failure of devices, from the switch. The fabric manager is equivalent to a centralized software-defined network controller as it controls the per-port forwarding and is aware of all the route changes. However, CXL fabric has (1) a limiting addressing scheme to support multi-path and adaptive routing, (2) single-path and inactive routing regardless of the traffic condition.

Challenge: Limited addressing support for multi-path and adaptive routing. The current PBR routing scheme assigns an ID to devices only. PBR routes packets to the destination device through multi-level switches with routing installed by the fabric manager. Any routing reconfiguration needs to go through the fabric manager and thus making load-aware, adaptive routing inefficient and infeasible on a large scale. The centralized routing of PBR prevents the usage of classic multi-pathing techniques like packet spraying or ECMP because all the routes are pre-determined by the fabric manager. Figure. 4 shows an example of multi-level CXL fabric.

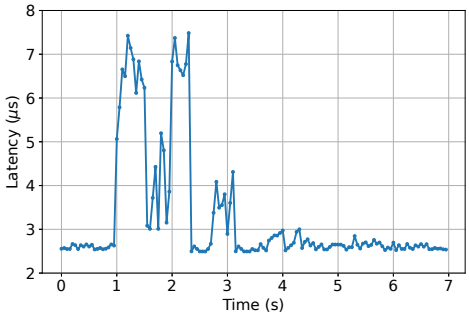


Figure 2: RDMA latency spikes almost 3 × by PCIe congestion.

Challenge: inflexible routing over diverse topologies. CXL fabric supports flexible topology and thus opens the possibility of having a wide range of topologies such as a fully connected graph, Fat-tree [16] or Dragonfly [29]. These topologies provide multiple paths for a source and a destination, but CXL fabric cannot route packets over multiple possible paths given its current design.

3.2 Transport-level Challenges

CXL inherits point-to-point flow control from PCIe that was designed for the communication between the device and CPU rather than a fabric. The flow control operates between two directly connected endpoints. They exchange credit tokens to evaluate the available buffer space on each side.

Challenges: Flow control cannot prevent congestion. The point-to-point, credit-based flow control is focused on not overrunning the buffer only. It cannot deal with pairs of endpoints sharing a port on the switch because no information is exchanged between them.

PCIe congestion experiment. We design an experiment of multiple flows sharing a port on a switch and creating congestion. Given no commercially available hardware for CXL, we use PCIe which shares the same flow control mechanism for our experiment. Interestingly, PCIe congestion has been identified and demonstrated under different setups [30, 34, 51].

We create artificial PCIe congestion on a PCIe switch port shared by two devices. The machine uses a SuperMicro X11SPA-TF motherboard with a Broadcom PEX8747 PCIe switch. The PCIe switch has one upstream port to the CPU, and two downstream ports connecting to the devices, An Nvidia 2080 Ti GPU and a ConnectX-5 RDMA NIC connected to the PCIe switch. The PCIe switch connects with the host processor on one side and

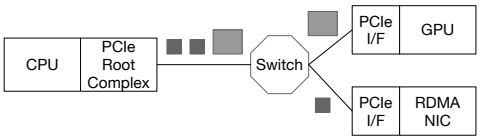


Figure 3: Experimental setup for PCIe congestion.

provides two ports connecting to GPU and RDMA NIC. The experimental setup is shown in Figure 3. During the experiment, the NIC periodically sends RDMA write requests to another machine every 100 microseconds. Each write request is sent from the CPU and travels through the PCIe switch to the NIC. After a second, a large integer array of 200 MB is moved from the main memory to the GPU. It causes heavy traffic on the upstream port and the downstream port to the GPU. Their traffic collides on the upstream port of the PCIe switch because flow control does not consider congestion that is caused by an outgoing link.

We use PerfTest of Linux-RDMA library to measure the RDMA write request latency [12] shown in Figure 2. The latency spikes to almost 3x from 2.593 μs to 7.483 μs at the peak of PCIe congestion. PCIe's virtual channel is a feasible but not scalable solution because it provides only 7 channels. Traffic on the same channel still suffers from the same congestion as we have shown above.

Challenge: Rate throttling between host CPU and devices only: CXL fabric offers a rate throttling mechanism called QoS Telemetry to avoid device overload and possible fabric congestion. The current QoS telemetry is designed specifically for CXL.mem between the host and devices with its local memory. QoS telemetry allows memory devices to indicate their current internal load with 2 bits for CXL.mem response packets. The sender on the host CPU uses the reported internal load to monitor and throttle its request rate. However, not every sender on CXL fabric is on the host CPU. Peer-to-peer memory accesses using Unordered I/O in CXL.io support devices accessing the memory of another device directly. These peer-to-peer accesses under CXL.io are not throttled in the current design because QoS telemetry is only for CXL.mem and does not consider the sender to be other than a host CPU. Machine learning and HPC applications illustrated in Figure 1 have evolved into heavy use of accelerators. Overlooking memory access from devices, especially accelerators, limits CXL fabric's ability to mitigate congestion and device overload.

Challenge: Inaccurate load & congestion information. QoS telemetry devises a mechanism called Egress Port Backpressure (EP Backpressure) to indicate the load

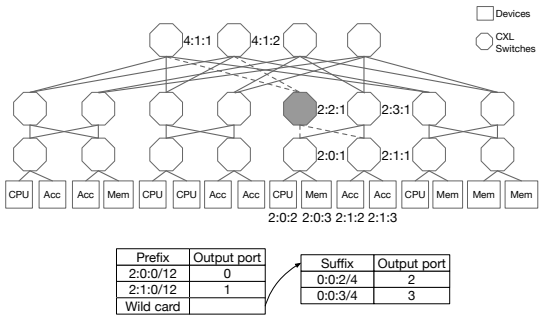


Figure 4: CXL fabric as a Fat-tree using 12-bit FAN-ID address *X:Y:Z*, which X, Y, and Z are hexadecimal values. Dash lines represent the routes from switch *2:2:1*'s routing table.

of CXL switch ports. It monitors the flow control backpressure situation on each CXL switch egress port. If the port cannot transmit packets due to insufficient credits, the port marks a 2-bit EP Backpressure value on the device load field of the outgoing request. The overall load of a device is determined by the maxima of the device's internal load and EP Backpressure.

However, the overall device load subsumes EP Backpressure, which is valuable congestion information. By taking the max value of the separated piece of information, QoS telemetry provides neither accurate device internal load nor congestion on the fabric. The inaccurate information prevents QoS telemetry from performing end-to-end flow control and congestion control for the transport protocol. The aforementioned challenges motivate us to design Aurelia.

4 DESIGN OF AURELIA

We propose Aurelia, a network design involving devices and switches on CXL fabric. Aurelia provides addressing, routing, and congestion control protocol design by augmenting necessary functionalities on the fabric interface and switches.

4.1 Addressing & Flow

Aurelia proposes to generalize CXL's port ID scheme to assign a 12-bit fabric node ID (FAN-ID) to a node that is either a device or a switch on the fabric. This is analogous to the 32-bit IP address describing hosts in an IP network. FAN-ID assignment is agnostic to the underlying topology as long as a FAN-ID is unique for a node on a CXL fabric. Currently, Aurelia assigns FAN-ID to the physical interface of the device. It does not assign FAN-ID to logical interfaces sharing the same physical interface.

With a similar analogy of "5-tuple" in the IP network, Aurelia defines a flow by a sequence of CXL packets that shares the same source device, destination device, protocol, and message classes. Thus, routable CXL packets can be identified with a quadruple: *(Source FAN-ID, Destination FAN-ID, CXL protocol, message classes)*. The CXL protocols include CXL.io, CXL.mem, and CXL.cache. The message classes are sub-type of packets within each protocol. For example, a CXL.mem packet writing to a device belongs to a class of *M2S Request with Data* and a CXL.cache packet that carries responses from the device to the host belongs to a class of *D2H Response with Data*.

4.2 Routing

Aurelia routes packets based on FAN-ID addresses like IP routing. CXL switches route a packet based on its routing table that maps a group of FAN-ID destinations onto a port. CXL switches transmit the packet out of a specific port to another switch that knows the next hop of this packet. The forwarding keeps on until the packet reaches the FAN-ID destination.

Routing: Fat-tree as an example. Constructing a Fat-tree [16] with 12-bit FAN-ID address shows a k-ary fat-tree with $k = 4$ in Figure 4. Fat-tree uses a hierarchical scheme that assigns FAN-ID to nodes as *pod:switch:device* with three segments. Each segment is a 4-bit exact, hexadecimal value. For example, the leftmost CPU has FAN-ID *2:0:2* representing it belongs to the third pod, the first switch, and the third node which is right after the switch itself. Like IP routing, the routing on Fat-tree takes a single shortest path despite that the Fat-tree topology provides path diversity. This creates bottlenecks even for trivial communication patterns because of the under-utilization of available bandwidth. Fat-tree implements a two-level routing table on each switch. One level of the table routes traffic down to the device and another one routes traffic toward the core of the fabric. The table maps a set of destinations to a specific port. The routing lookup of destinations uses the address prefix for the traffic downward to the devices and the address suffix for the traffic going toward the cores. The address suffix approach spreads traffic toward the fat-tree core across different switches based on FAN-ID. The bottom of Figure 4 shows the routing table of CXL switch *2:2:1* filled in gray. The prefix table routes packets toward its downstream switches *2:0:1* and *2:1:1*. The suffix table routes the packet upward to core switches *4:1:1* and *4:1:2*.

Multi-path routing. The two-level routing table of Fat-tree is just an implementation of multi-path routing together with a specific topology. Aurelia imposes no restriction on how the fabric should be constructed.

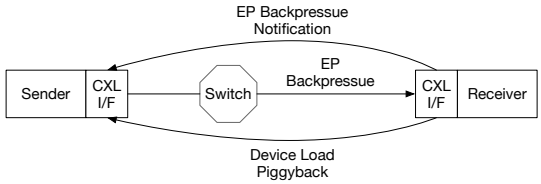


Figure 5: Aurelia has a congestion control loop for the fabric, a flow-control loop for the device.

Instead, Aurelia requires routing tables to be able to map a destination to multiple next-hop FAN-ID and its corresponding metric. The metric can be distances in terms of hops or local congestion level on each switch port. Aurelia selects a next-hop with minimal metric and randomly selects one if there are multiple next-hop options with equal metric. Considering the number of hops for shortest path routing, Aurelia can perform a random selection for the next hop on flow granularity or on packet granularity. The former is similar to Equal-Cost Multipathing (ECMP) and the latter is similar to packet spraying.

Adaptive routing. ECMP and packet spraying are oblivious to the workload. Aurelia can further exploit local per-port information on the switches or the fabric manager manipulating the routing table to achieve adaptive routing. Aurelia uses per-port EP Backpressure as a measure of local congestion indication. For selecting an egress port with minimal congestion on the switch, Aurelia uses power-of-2 choice [38] to avoid congestion on ports based on possibly delayed congestion information. Additionally, Aurelia allows the fabric manager to insert routes and has the sole authority to modify the routing table at any time. This is useful for the workload that requires non-trivial routes tailored to specific workloads [53, 54].

4.3 End-to-end Congestion Control

Aurelia implements end-to-end congestion control with overload avoidance by extending existing mechanisms of CXL. The existing mechanisms are EP Backpressure, rate throttling, and internal load reported by devices. EP Backpressure describes the queuing situation in a similar way as Explicit Congestion Notification (ECN) does on Ethernet and Infiniband. EP Backpressure and ECN both indicate congestion explicitly to the receiving endpoints and need packets to convoy the congestion signal back to the sender. Rate throttling controls the data injection rate into the fabric and thus avoids congestion. Internal load is used to avoid overloading device that is likely to cause additional device-side delay or loss

of CXL packets. These factors determine Aurelia to implement a rate-based congestion control with overload avoidance for the devices.

Aurelia separates EP Backpressure and the device's internal load for congestion control and overload avoidance. EP Backpressure as a 2-bit congestion signal on the fabric is piggybacked to the sender if its value is larger than a threshold. The 2-bit device internal load serves a quantized credit for the sender not exceeding the receiver's processing rate. Using credit to regulate sending rate effectively achieves end-to-end flow control to avoid overloading. Flow control preventing buffer-overrun is handled by the link and physical layer of CXL and thus not a concern for the transport layer. Also, Aurelia generalizes the reporting of device internal load for all memory access, including peer-to-peer memory access between devices.

Peer-to-peer memory access between devices without embedded CPU motivates the need of implementing rate throttling logic on CXL interface hardware. Also, rate throttling has a tight latency requirement of sub-μs on CXL fabric. Pond measured end-to-end CXL.mem latency and obtained latency ranging to 270 ns on CXL system with a single switch [33, 49]. Using Pond's latency assumption, the latency of CXL packet traveling through a two-level fat-tree takes around 680 ns.

Combining sub-μs latency and peer-to-peer memory access requirements, Aurelia relies on hardware to throttle rate for congestion control. The configuration of these hardware is kept on the host CPU, but the actual rate throttling is on hardware to meet these requirements. Implementing rate throttling logic on hardware has been used on RDMA over Infiniband and Ethernet with RoCEv2 support. Hardware-based rate throttling ensures all devices on the CXL fabric are able to control their injection rate and further regulate the congestion. Before throttling, Aurelia sets the rate as the target rate for later recovery. Aurelia throttles the rate of the sender, when the receiver notifies it of the congestion signal and generates EP Backpressure. After rate throttling, Aurelia enters the recovery phase and then performs additive increase when approaching the target rate [58].

5 DISCUSSION

Alternatives: NVLink and PCIe fabric. NVLink is an interconnect by Nvidia for high throughput between GPUs [11]. NVLink supports GPU-CPU interconnect with cache coherency on the recent Grace-Hopper 200 hardware [5]. NVLink presents an interesting alternative to CXL but has three limitations. First, NVLink with its cache coherent extension in fact supports GPUs as CXL type 2 devices. However, it does not support CXL type 3 devices that expand memory independent of the main memory capacity. The capability to scale memory capacity is attractive for memory-hungry large language models [19, 52]. Second, NVLink scales to 256 endpoints though connects only GPUs as endpoints [6]. Third, NVLink as a propriety interconnect limits its usage beyond Nvidia's hardware. PCIe using Non-Transparent Bridge (NTB) expands beyond a single root complex to multiple hosts as a fabric with many more connected devices [13]. However, PCIe, included in CXL as CXL.io, lacks the memory and caching semantics that are much needed in the face of accelerator and memory expansion. This is exactly the reason that motivates the creation of CXL. Both NVLink and PCIe fabric show a fabric supporting a subset of CXL's use cases but not all of it.
Cost-effective scales of CXL fabric: CXL is an exciting technology but it comes with its limitations. First, CXL requires retimer to maintain its signal integrity after 500 mm distance [4, 15]. The retimer raises the hardware cost and incurs extra 20 ns latency every 500 [33]. Second, to scale CXL supporting multi-level switching, multiple CXL switches are used. They cause an estimated 70 ns latency for each hop over a switch. Considering the combined cost of switches, retimers and additional memory controllers, Pond [18] suggests a scale smaller than 32-socket for their memory expansion usage (Shown in Figure 1b). However, the exact scale of cost-effective CXL fabric for model training and HPC usage (Shown in Figure 1a) remains to be determined in future works.
Security implication: Delay based side channel. CXL fabric exposes the server interconnect to a shared fabric that may contain malicious devices. The fabric exposes a delay-based side channel caused by interconnect congestion. This side channel enables attackers to recover information from different delay patterns. Previous works [51] investigate the same issue with PCIe on a server. CXL fabric enlarges the attack interface by leaving all devices vulnerable to delay probing. Defense against this specific type of side channel attack is interesting for future security research.

6 CONCLUSION

Emerging standard CXL facilitates disaggregation with the support of multi-level switching. However, the current CXL fabric presents scalability and latency limitations. Aurelia addresses the challenges by designing proper addressing, routing, and transport layer design.

REFERENCES

[1] Characterization of the doe mini-apps. https://portal.nersc.gov/project/CAL/doe-miniapps.htm.

[2] Crossroads benchmarks. https://www.lanl.gov/projects/crossroads/benchmarks-performance-analysis.php.

[3] Cxl 3.0 specification. https://www.computeexpresslink.org/download-the-specification.

[4] Cxl retimer. https://www.microchip.com/en-us/about/media-center/blog/2020/cxl--use-cases-driving-the-need-for-low-latency-performance-reti.

[5] DGX GH200. https://www.nvidia.com/en-us/data-center/dgx-gh200/.

[6] DGX SuperPOD. https://www.nvidia.com/en-us/data-center/dgx-superpod/.

[7] Dram resource scalability enabled by cxl. https://www.computeexpresslink.org/post/dram-resource-scalability-enabled-by-cxl.

[8] Ecp proxy applications. https://proxyapps.exascaleproject.org/app/.

[9] Ember. https://proxyapps.exascaleproject.org/app/ember-communication-patterns/.

[10] Fabric saving for gpu. https://gigaio.com/2023/01/gigaio-doubles-gpu-performance-at-a-30-cost-savings-with-intel-sapphire-rapids/.

[11] NVLink. https://www.nvidia.com/en-us/data-center/nvlink/.

[12] Open fabrics enterprise distribution (ofed) performance tests. https://github.com/linux-rdma/perftest.

[13] Pci-sig specifications. https://pcisig.com/specifications.

[14] Pcie 6.0 and 7.0. https://www.xda-developers.com/pcie-6-to-launch-in-2024-pcie-7-in-2027/.

[15] Pcie retimer. https://www.asteralabs.com/smart-retimers/pci-express-retimers-vs-redrivers-an-eye-popping-difference/.

[16] Mohammad Al-Fares, Alexander Loukissas, and Amin Vahdat. A scalable, commodity data center network architecture. In *ACM Conference on Special Interest Group on Data Communication*, 2008.

[17] Emmanuel Amaro, Christopher Branner-Augmon, Zhihong Luo, Amy Ousterhout, Marcos K. Aguilera, Aurojit Panda, Sylvia Ratnasamy, and Scott Shenker. Can far memory improve job throughput? In *Proceedings of the Fifteenth European Conference on Computer Systems*, 2020.

[18] Daniel S. Berger, Daniel Ernst, Huaicheng Li, Pantea Zardoshti, Monish Shah, Samir Rajadnya, Scott Lee, Lisa Hsu, Ishwar Agarwal, Mark D. Hill, and Ricardo Bianchini. Design tradeoffs in cxl-based memory pools for public cloud platforms. *IEEE Micro*, 2023.

[19] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, 2020.

[20] Irina Calciu, M. Talha Imran, Ivan Puddu, Sanidhya Kashyap, Hasan Al Maruf, Onur Mutlu, and Aasheesh Kolli. Rethinking software runtimes for disaggregated memory. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021.

[21] Zhichao Cao and Siying Dong. Characterizing, modeling, and benchmarking rocksdb key-value workloads at facebook. In *Proceedings of the 18th USENIX Conference on File and Storage Technologies*, 2020.

[22] Nan Ding, Pieter Maris, Hai Ah Nam, Taylor Groves, Muaaz Gul Awan, LeAnn Lindsey, Christopher Daley, Oguz Selvitopi, Leonid Oliker, Nicholas Wright, and Samuel Williams. Evaluating the potential of disaggregated memory systems for hpc applications, 2023.

[23] Aleksandar Dragojević, Dushyanth Narayanan, Miguel Castro, and Orion Hodson. FaRM: Fast remote memory. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*, April 2014.

[24] Donghyun Gouk, Sangwon Lee, Miryeong Kwon, and Myoungsoo Jung. Direct access, High-Performance memory disaggregation with DirectCXL. In *USENIX Annual Technical Conference (USENIX ATC)*, 2022.

[25] Simon David Hammond. Compute memory trends: from application requirements to architectural needs. 2020.

[26] Kostis Kaffes, Timothy Chong, Jack Tigar Humphries, Adam Belay, David Mazières, and Christos Kozyrakis. Shinjuku: Preemptive scheduling for usecond-scale tail latency. In *USENIX Conference on Networked Systems Design and Implementation (NSDI)*, 2019.

[27] Anuj Kalia, Michael Kaminsky, and David Andersen. Datacenter RPCs can be general and fast. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, February 2019.

[28] Anuj Kalia, Michael Kaminsky, and David G. Andersen. Using rdma efficiently for key-value services. In *Proceedings of the 2014 ACM Conference on SIGCOMM*, 2014.

[29] John Kim, Wiliam J. Dally, Steve Scott, and Dennis Abts. Technology-driven, highly-scalable dragonfly topology. In *International Symposium on Computer Architecture (ISCA)*, 2008.

[30] V. Krishnan and D. Mayhew. Localized congestion control in advanced switching interconnects. *IEEE Micro*, 25(1):10–11, 2005.

[31] Ignacio Laguna, Ryan Marshall, Kathryn Mohror, Martin Ruefenacht, Anthony Skjellum, and Nawrin Sultana. A large-scale study of mpi usage in open-source hpc applications. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2019.

[32] Youngmoon Lee, Hasan Al Maruf, Mosharaf Chowdhury, Asaf Cidon, and Kang G. Shin. Hydra : Resilient and highly available remote memory. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*, February 2022.

[33] Huaicheng Li, Daniel S. Berger, Stanko Novakovic, Lisa Hsu, Dan Ernst, Pantea Zardoshti, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. Pond: CXL-Based Memory Pooling Systems for Cloud Platforms. In *To be appeared in ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[34] Maxime Martinasso, Grzegorz Kwasniewski, Sadaf R. Alam, Thomas C. Schulthess, and Torsten Hoefler. A pcie congestion-aware performance model for densely populated accelerator servers. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2016.

[35] Michael Marty, Marc de Kruijf, Jacob Adriaens, Christopher Alfeld, Sean Bauer, Carlo Contavalli, Mike Dalton, Nandita

Dukkipati, William C. Evans, Steve Gribble, Nicholas Kidd, Roman Kononov, Gautam Kumar, Carl Mauer, Emily Musick, Lena Olson, Mike Ryan, Erik Rubow, Kevin Springborn, Paul Turner, Valas Valancius, Xi Wang, and Amin Vahdat. Snap: a microkernel approach to host networking. In *ACM SIGOPS 27th Symposium on Operating Systems Principles*, 2019.

[36] Hasan Al Maruf and Mosharaf Chowdhury. Effectively prefetching remote memory with leap. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, July 2020.

[37] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit Kanaujia, and Prakash Chauhan. Tpp: Transparent page placement for cxl-enabled tiered-memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2023.

[38] M. Mitzenmacher. The power of two choices in randomized load balancing. *IEEE Transactions on Parallel and Distributed Systems*, 12(10):1094–1104, 2001.

[39] Dheevatsa Mudigere, Yuchen Hao, Jianyu Huang, Zhihao Jia, Andrew Tulloch, Srinivas Sridharan, Xing Liu, Mustafa Ozdal, Jade Nie, Jongsoo Park, Liang Luo, Jie (Amy) Yang, Leon Gao, Dmytro Ivchenko, Aarti Basant, Yuxi Hu, Jiyan Yang, Ehsan K. Ardestani, Xiaodong Wang, Rakesh Komuravelli, Ching-Hsiang Chu, Serhat Yilmaz, Huayu Li, Jiyuan Qian, Zhuobo Feng, Yinbin Ma, Junjie Yang, Ellie Wen, Hong Li, Lin Yang, Chonglin Sun, Whitney Zhao, Dimitry Melts, Krishna Dhulipala, KR Kishore, Tyler Graf, Assaf Eisenman, Kiran Kumar Matam, Adi Gangidi, Guoqiang Jerry Chen, Manoj Krishnan, Avinash Nayak, Krishnakumar Nair, Bharath Muthiah, Mahmoud khorashadi, Pallab Bhattacharya, Petr Lapukhov, Maxim Naumov, Ajit Mathews, Lin Qiao, Mikhail Smelyanskiy, Bill Jia, and Vijay Rao. Software-hardware co-design for fast and scalable training of deep learning recommendation models. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022.

[40] Rolf Neugebauer, Gianni Antichi, José Fernando Zazo, Yury Audzevich, Sergio López-Buedo, and Andrew W. Moore. Understanding pcie performance for end host networking. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, 2018.

[41] Rajesh Nishtala, Hans Fugal, Steven Grimm, Marc Kwiatkowski, Herman Lee, Harry C. Li, Ryan McElroy, Mike Paleczny, Daniel Peek, Paul Saab, David Stafford, Tony Tung, and Venkateshwaran Venkataramani. Scaling memcache at facebook. In *10th USENIX Symposium on Networked Systems Design and Implementation*, 2013.

[42] Amy Ousterhout, Joshua Fried, Jonathan Behrens, Adam Belay, and Hari Balakrishnan. Shenango: Achieving high cpu efficiency for latency-sensitive datacenter workloads. In *USENIX Conference on Networked Systems Design and Implementation (NSDI)*, 2019.

[43] S. J. Park, H. Kim, K.-S. Kim, J. So, J. Ahn, W.-J. Lee, D. Kim, Y.-J. Kim, J. Seok, J.-G. Lee, H.-Y. Ryu, C. Y. Lee, J. Prout, K.-C. Ryoo, S.-J. Han, M.-K. Kook, J. S. Choi, J. Gim, Y. S. Ki, S. Ryu, C. Park, D.-G. Lee, J. Cho, H. Song, and J. Y. Lee. Scaling of memory performance and capacity with cxl memory expander. In *IEEE Hot Chips 34 Symposium (HCS)*, 2022.

[44] Milan Radulovic. *Memory bandwidth and latency in HPC: system requirements and performance impact.* PhD thesis, Polytechnic University of Catalonia, Spain, 2019.

[45] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models, 2020.

[46] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021.

[47] Zhenyuan Ruan, Malte Schwarzkopf, Marcos K. Aguilera, and Adam Belay. AIFM: High-Performance, Application-Integrated far memory. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020.

[48] Yizhou Shan, Yutong Huang, Yilun Chen, and Yiying Zhang. LegoOS: A disseminated, distributed OS for hardware resource disaggregation. In *OSDI*, 2018.

[49] Debendra Das Sharma. Compute express link®: An open industry-standard interconnect enabling heterogeneous data-centric computing. In *IEEE Symposium on High-Performance Interconnects (HOTI)*, 2022.

[50] Debendra Das Sharma. Novel composable and scale-out architectures using compute express link. *IEEE Micro*, 2023.

[51] Mingtian Tan, Junpeng Wan, Zhe Zhou, and Zhou Li. Invisible probe: Timing attacks with pcie congestion side-channel. In *IEEE Symposium on Security and Privacy (S&P)*, 2021.

[52] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.

[53] Hung-Wei Tseng, Yang Liu, Mark Gahagan, Jing Li, Yanqin Jing, and Steven Swanson. Gullfoss: Accelerating and simplifying data movement among heterogeneous computing and storage resources. Technical report, UC San Diego, 2015.

[54] Lluís Vilanova, Lina Maudlej, Shai Bergman, Till Miemietz, Matthias Hille, Nils Asmussen, Michael Roitzsch, Hermann Härtig, and Mark Silberstein. Slashing the disaggregation tax in heterogeneous data centers with fractos. In *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys)*, 2022.

[55] Chenxi Wang, Yifan Qiao, Haoran Ma, Shi Liu, Wenguang Chen, Ravi Netravali, Miryung Kim, and Guoqing Harry Xu. Canvas: Isolated and adaptive swapping for Multi-Applications on remote memory. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, April 2023.

[56] Xingda Wei, Rong Chen, and Haibo Chen. Fast RDMA-based ordered Key-Value store using remote learned cache. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, November 2020.

[57] Yang Zhou, Hassan M. G. Wassel, Sihang Liu, Jiaqi Gao, James Mickens, Minlan Yu, Chris Kennelly, Paul Turner, David E. Culler, Henry M. Levy, and Amin Vahdat. Carbink: Fault-Tolerant far memory. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, July 2022.

[58] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. Congestion control for large-scale rdma deployments. In *ACM Conference on Special Interest Group on Data Communication*, 2015.